

SIMATS ENGINEERING
Department of Computer Science and Engineering ASSESSMENT
TOOL 1 – EXPERIMENTS

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO3 – Precision (BL3)	Assessment:	Experiments
Weightage:	45%	Total Marks:	100
CO3 Statement:	Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT).		
Student Name:	Moniga V	Reg. No.:	192521184
Section / Batch:	2025	Date:	10.08.26

Instructions to Students

- Identify the relevant concepts of register transfers, ALU operations, bus organization, pipelining, hazards, and superscalar processing.
- Analyze the execution of data transfers, arithmetic and logic operations, instruction processing, and parallel execution.
- Apply suitable techniques such as multiple buses, pipelining, stalling, forwarding, branch prediction, and speculative execution.
- Compute performance measures such as execution time, clock cycles, speedup, throughput, and resource utilization.
- Interpret the results by evaluating performance improvements, bottlenecks, and the suitability of each architectural technique.

QUESTIONS

1. Design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. Use multiple registers to demonstrate how data is transferred between them during execution, and analyze the sequence of operations and the role of the ALU in processing data.

Aim

To design and simulate a processor system that performs register transfer operations along with arithmetic and logical operations using multiple registers and an ALU.

System Components

- Multiple registers such as R1, R2, R3, and R4
- Arithmetic Logic Unit (ALU)
- Multiplexer
- Common data path
- Control unit
- Control signals

Register Transfer Operations

Register transfer operations move data from one register to another.

Examples:

- $R2 \leftarrow R1$ – transfers the contents of R1 to R2.

- **R3 ← R2** – transfers the contents of R2 to R3.
- **R4 ← R1** – transfers the contents of R1 to R4.

The control unit selects the source register and destination register and enables the required transfer.

Arithmetic Operations

Addition:

The ALU receives the contents of R1 and R2, performs addition, and stores the result in R3.

Subtraction:

The ALU subtracts the contents of R2 from R1 and stores the result in R3.

Logic Operations

AND operation:

The ALU performs a bit-by-bit AND operation.

OR operation:

The ALU performs a bit-by-bit OR operation.

Execution Sequence

1. Load the required values into the registers.
2. Select the source register using control signals.
3. Transfer the operands to the ALU.
4. Select the required ALU operation.
5. Perform the arithmetic or logical operation.
6. Transfer the result to the destination register.
7. Verify the result stored in the destination register.

Role of ALU

The ALU is the main processing unit responsible for arithmetic and logical operations. It receives operands from the registers and performs the operation selected by the control unit. The result is then transferred back to a register.

Result

The system successfully performs register transfers, addition, subtraction, AND, and OR operations. The experiment demonstrates how registers, the ALU, and control signals work together during processor execution.

2. Create a model of a computer system using single-bus and multi-bus organization, and perform data transfer operations between registers, memory, and I/O. Compare the time taken and efficiency of both approaches, and analyse how multiple bus structures improve parallel data transfer and reduce bottlenecks.

Aim

To model a computer system using single-bus and multi-bus organizations and compare their data-transfer performance.

Single-Bus Organization

In a single-bus organization, CPU registers, memory, and I/O devices share one common bus for data transfer.

Only one major data transfer can use the common bus at a time. Therefore, when multiple registers or devices need the bus simultaneously, they must wait for access.

For example:

R1 → Bus → R2

The contents of R1 are placed on the bus and then loaded into R2.

Multi-Bus Organization

In a multi-bus organization, two or more buses are available for transferring data.

For example, two source registers can provide operands simultaneously to the ALU while another path can be used to transfer the result to a destination register.

This allows multiple data transfers to occur during the same clock cycle.

Data Transfer Operations

The following operations can be performed:

- Register-to-register transfer
- Register-to-memory transfer
- Memory-to-register transfer
- I/O-to-register transfer
- Register-to-I/O transfer

Comparison

FEATURE	SINGLE-BUS	MULTI-BUS
NUMBER OF BUSES	One	Two or more
DATA TRANSFER	Mostly sequential	More parallel
HARDWARE COMPLEXITY	Low	Higher
COST	Lower	Higher
DATA-TRANSFER SPEED	Lower	Higher
BUS CONFLICTS	More frequent	Reduced
PARALLEL OPERATIONS	Limited	Greater
BOTTLENECKS	More likely	Less likely

Efficiency Analysis

The single-bus system is simpler and requires fewer hardware components, but frequent bus sharing creates bottlenecks. When several operations require the bus, some operations have to wait.

The multi-bus system provides additional paths for data movement. This allows multiple operands to be transferred simultaneously and reduces the time required for complex operations.

Result

The multi-bus organization provides better performance and higher data-transfer efficiency than the single-bus organization because it supports greater parallelism and reduces bus contention.

3. Simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back) and execute a set of instructions. Measure the performance in terms of speedup and throughput, and compare it with a non-pipelined system to evaluate the effectiveness of pipelining.

Aim

To simulate a five-stage instruction pipeline and compare its performance with a non-pipelined processor.

Pipeline Stages

The processor uses five stages:

- **Fetch (IF):** Fetches the instruction from memory.
- **Decode (ID):** Decodes the instruction and identifies operands.
- **Execute (EX):** Performs the required operation.
- **Memory (MEM):** Performs memory access when required.
- **Write-back (WB):** Stores the result in the destination register.

Instruction Set

Consider five instructions:

- **I1:** ADD R1, R2, R3
- **I2:** SUB R4, R5, R6
- **I3:** LOAD R7, 0(R1)
- **I4:** ADD R8, R7, R9
- **I5:** SUB R10, R11, R12

Non-Pipelined Execution

In a non-pipelined processor, each instruction completes all five stages before the next instruction begins.

For 5 instructions:

Pipelined Execution

In a pipelined processor, different instructions can occupy different stages simultaneously.

For **n instructions** and **k stages**:

For 5 instructions and 5 stages:

Speedup

Therefore, the ideal speedup is approximately **2.78 times**.

Throughput

Throughput is the number of instructions completed per unit of time.

After the pipeline becomes full, ideally one instruction can be completed during every clock cycle.

Therefore, pipelining increases instruction throughput compared with sequential execution.

Performance Factors

Actual performance can be affected by:

- Data hazards
- Branch hazards
- Structural hazards
- Pipeline stalls
- Memory delays
- Cache misses

Result

The pipelined processor completes the given instruction sequence in fewer cycles than the non-pipelined processor. The experiment demonstrates that instruction pipelining improves throughput and reduces overall execution time.

4. Develop a pipeline execution scenario where data hazards, control hazards, and structural hazards occur. Observe their impact on instruction execution, and implement techniques such as stalling, data forwarding, and branch prediction to resolve these hazards and improve performance.

Aim

To study data, control, and structural hazards in a pipeline and implement techniques to reduce their performance impact.

Data Hazard

A data hazard occurs when one instruction depends on the result of a previous instruction.

Example:

- **I1:** ADD R1, R2, R3
- **I2:** SUB R4, R1, R5

I2 requires the value of R1 produced by I1. If the value is not available when I2 needs it, a data hazard occurs.

Control Hazard

A control hazard occurs because of branch instructions.

Example:

- **I1:** ADD R1, R2, R3
- **I2:** BEQ R1, R0, TARGET

The processor does not immediately know which instruction should be fetched after the branch. This can cause incorrect instructions to enter the pipeline.

Structural Hazard

A structural hazard occurs when two instructions require the same hardware resource at the same time. For example, if instruction fetching and a memory-access instruction both require the same memory unit, one instruction may have to wait.

Hazard Resolution Techniques

1. Pipeline Stalling

The processor temporarily stops the dependent instruction until the required data or resource becomes available.

Advantage:

- Prevents incorrect execution.

Disadvantage:

- Increases execution time.

2. Data Forwarding

Data forwarding sends the result directly from one pipeline stage to another without waiting for the normal register write-back operation.

Advantage:

- Reduces the number of stalls.
- Improves pipeline performance.

3. Branch Prediction

The processor predicts the outcome of a branch before the actual result is available.

If the prediction is correct, pipeline execution continues normally. If the prediction is incorrect, the incorrectly fetched instructions are removed.

Performance Analysis

Without hazard-handling techniques, frequent dependencies and branches can cause many pipeline stalls. Forwarding reduces data-related delays, while branch prediction reduces control-related delays. Structural hazards can be reduced by providing additional hardware resources or separate instruction and data paths.

Result

The experiment demonstrates that pipeline hazards can significantly reduce processor performance. Stalling provides correctness, while forwarding and branch prediction reduce unnecessary delays and improve instruction throughput.

5. Analyse a processor supporting superscalar execution, including out-of-order and speculative execution, and extend the study to include hyper-threading and simultaneous multithreading (SMT). Evaluate how multiple instructions are executed in parallel and compare the performance improvements in terms of instruction throughput and resource utilization.

Aim

To analyse a processor that supports superscalar execution, out-of-order execution, speculative execution, hyper-threading, and simultaneous multithreading.

Superscalar Execution

A superscalar processor can execute multiple instructions during the same clock cycle.

For example, if a processor has two suitable execution units, two independent instructions can potentially execute simultaneously.

This increases **instruction-level parallelism** and improves instruction throughput.

Out-of-Order Execution

Out-of-order execution allows instructions to execute when their operands and required resources are available instead of strictly following program order.

For example, if one instruction is waiting for data from memory, another independent instruction can execute first.

This prevents processor resources from remaining idle.

Speculative Execution

Speculative execution allows the processor to execute instructions before it is certain that those instructions are required.

It is commonly combined with branch prediction. If the prediction is correct, useful work has already been completed. If the prediction is incorrect, the speculative results are discarded.

Hyper-Threading

Hyper-threading is a technology that allows a single physical processor core to appear as multiple logical processors.

The logical processors share many physical resources but can execute instructions from different threads. When one thread is waiting, another thread can use available processor resources.

Simultaneous Multithreading (SMT)

SMT allows instructions from multiple threads to be issued and executed simultaneously on the same processor core.

It improves resource utilization by allowing different threads to use available execution units.

Resource Utilization

Important processor resources include:

- Integer ALUs
- Floating-point units
- Load/store units
- Registers
- Cache
- Instruction scheduler
- Reorder buffer

Superscalar execution increases parallel instruction execution, while out-of-order execution helps keep execution units busy. SMT allows unused resources to be utilized by another thread.

Performance Comparison

EXECUTION METHOD	MAIN BENEFIT
SUPERSCALAR	Multiple instructions per cycle
OUT-OF-ORDER	Reduces waiting caused by dependencies
SPECULATIVE	Reduces branch-related delays
HYPER-THREADING	Allows multiple logical threads
SMT	Improves utilization of processor resources

Performance Measures

Instruction Throughput:

Measures the number of instructions completed per unit of time. Superscalar execution and SMT can increase throughput when sufficient independent instructions are available.

Resource Utilization:

Measures how effectively processor execution units are being used. Out-of-order execution and SMT can improve utilization by filling otherwise idle execution slots.

Result

A processor combining superscalar, out-of-order, speculative execution, and SMT can execute multiple instructions from one or more threads simultaneously. These techniques improve instruction throughput and processor resource utilization, although the actual performance depends on instruction dependencies, branch prediction accuracy, memory latency, and available hardware resources.

Code of Conduct

I MONIGA V (192521184) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation

Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Marks Evaluation:

Q. No	Understanding of Concepts(25)	Experimental Design (30)	Use of Tools(20)	Analysis of Results (15)	Documentation (10)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/ 30	/ 20	/ 15	/ 10	/ 100

Course Coordinator

Faculty Incharge